

CubeSat ADCS Hardware-in-the-Loop Development Plan

Caleb Oblepias
Vanderbilt University
March 26, 2026

Contents

1	Development Overview	2
2	System Overview	2
2.1	Architectural Layers	2
3	Development Methodology	3
3.1	Phase 1: Python Truth Model Development	3
3.1.1	Dynamics and Environment Rigid-Body Simulator	3
3.1.2	State Estimator	3
3.1.3	Guidance and Control Law	4
3.1.4	Unit Testing Strategy	4
3.1.5	Outputs Logged Per Timestep	4
3.2	Phase 2: FPGA Porting and Verification	4
3.2.1	Objectives	4
3.2.2	Fixed-Point Strategy	5
3.2.3	Verification Approach	5
3.3	Phase 3: RTOS-Based Interface Layer and OBC Development	5
3.3.1	Algorithm Porting and Verification	5
3.3.2	OBC RTOS Task Architecture	5
3.3.3	Real-Time Interface Layer	6
3.3.4	Deterministic Timing Requirements	7
3.4	Phase 4: Full Hardware-in-the-Loop Integration	7
3.4.1	Integration Levels	7
3.4.2	Logged Signals	7
4	Interface Control Document (ICD)	7
4.1	IMU Packet Structure	7
4.2	Torque Command Packet	8

1 Development Overview

This project follows a development strategy designed to ensure correctness before real-time and hardware constraints are introduced.

Module development proceeds in the following order:

1. **Python Rigid-Body Dynamics and Environment Simulator**
2. **Python State Estimator**
3. **Python Guidance and Control Law**
4. **Closed-Loop Python Validation**
5. **FPGA Port of Dynamics and Sensor Formatting**
6. **C Port of Estimator and Control Law**
7. **RTOS Integration on OBC MCU**
8. **Real-Time Interface Layer Implementation on HIL MCU**
9. **Full Hardware-in-the-Loop Integration**

At each stage, the previously validated implementation serves as the verification reference for the next hardware or language transition. The Python truth model functions as the authoritative reference throughout all subsequent development phases.

2 System Overview

This document defines the development, verification, and integration plan for a CubeSat Attitude Determination and Control System (ADCS) Hardware-in-the-Loop (HIL) platform.

The system consists of three major computational modules:

- Python Truth Model (reference simulation)
- FPGA Real-Time Dynamics and Sensor Simulation
- OBC Flight Software (RTOS-based MCU)

The objective is to develop a deterministic, verifiable closed-loop ADCS architecture with traceable validation against a trusted Python simulation model.

2.1 Architectural Layers

- **Dynamics and Environment Simulator** (Python $\rightarrow$ FPGA)
- **State Estimation** (Python $\rightarrow$ RTOS OBC)
- **Guidance and Control Law** (Python $\rightarrow$ RTOS OBC)
- **Real-Time Interface Layer** (RTOS MCU)

3 Development Methodology

The system will be developed in four primary phases to ensure deterministic validation and controlled integration.

3.1 Phase 1: Python Truth Model Development

The closed-loop ADCS simulation is structured into three primary functional modules:

1. Dynamics and Environment Rigid-Body Simulator
2. State Estimator
3. Guidance and Control Law

Each module operates at a fixed timestep and exchanges well-defined data structures to ensure deterministic integration and traceable verification.

3.1.1 Dynamics and Environment Rigid-Body Simulator

Purpose: Simulate the rotational motion of the spacecraft under applied control torques and environmental disturbances using a rigid-body dynamics model.

Inputs:

- Actuator torque commands (from Guidance/Control Law)
- External disturbance parameters (e.g., gravity gradient, magnetic field, aerodynamic drag)
- Initial attitude and angular velocity
- Fixed timestep

Outputs:

- Truth attitude state (quaternion)
- Truth angular velocity
- Truth angular acceleration

This module represents the physical spacecraft and environment and serves as the authoritative truth reference for system verification.

3.1.2 State Estimator

Purpose: Estimate the spacecraft attitude and angular velocity using simulated or hardware-generated sensor measurements.

Inputs:

- Sensor measurements
- Previous state estimate
- Timestamp information

Outputs:

- Estimated attitude
- Estimated angular velocity

The estimator reconstructs the spacecraft state from imperfect sensor data and provides the control-relevant state to the Guidance and Control module.

3.1.3 Guidance and Control Law

Purpose: Compute actuator torque commands required to drive the spacecraft from its current estimated state to a desired attitude.

Inputs:

- Estimated attitude and angular velocity (from State Estimator)
- Desired attitude reference
- Control gains and configuration parameters

Outputs:

- Torque command vector
- Optional: actuator saturation flags or control mode indicators

The torque command is fed back into the Dynamics Simulator, closing the loop.

3.1.4 Unit Testing Strategy

- Zero torque $\rightarrow$ angular momentum conserved.
- Constant torque $\rightarrow$ matches analytical solution.
- Quaternion norm remains unity.

3.1.5 Outputs Logged Per Timestep

- Truth State
- Sensor Output
- Estimated State
- Controller Output
- Simulation Timestamp

3.2 Phase 2: FPGA Porting and Verification

3.2.1 Objectives

- Port dynamics and sensor formatting to fixed-point implementation.
- Establish deterministic real-time execution.
- Validate FPGA outputs against Python truth model.

3.2.2 Fixed-Point Strategy

- Define scaling factors for angular rate, quaternion elements, torque.
- Perform overflow and saturation analysis.
- Validate numerical stability over extended simulation duration.

3.2.3 Verification Approach

For identical torque input sequences:

$$\|x_{\text{FPGA}} - x_{\text{Python}}\| < \epsilon \quad (1)$$

3.3 Phase 3: RTOS-Based Interface Layer and OBC Development

Phase 3 transitions the estimator and control law from Python into a real-time embedded implementation and establishes the deterministic interface between hardware modules.

This phase is divided into three structured sub-stages:

1. Algorithm Porting and Verification in C
2. Real-Time Task Architecture Implementation
3. Hardware Interface and Deterministic Timing Validation

3.3.1 Algorithm Porting and Verification

The State Estimator and Guidance/Control Law modules will be reimplemented in C.

Objectives:

- Ensure numerical equivalence with Python reference implementation.
- Validate deterministic behavior under fixed timestep execution.

Verification Method:

- Feed identical logged sensor data from Phase 1 into the C implementation.
- Compare estimated state and torque outputs against Python outputs.
- Acceptable deviation must remain within defined numerical tolerance.

This stage isolates algorithm correctness from real-time scheduling complexity.

3.3.2 OBC RTOS Task Architecture

After algorithm validation, modules are integrated into an RTOS-based task structure.

Primary Tasks:

- Sensor Acquisition Task
- State Estimation Task
- Control Law Task

- Actuator Command Task
- Telemetry and Logging Task

Unit Testing at RTOS Level:

- Validate task execution frequency using hardware timers.
- Measure worst-case execution time (WCET) per task.

3.3.3 Real-Time Interface Layer

The real-time interface MCU manages communication between:

- FPGA dynamics engine
- OBC MCU

Responsibilities:

- Timestamp propagation
- Deterministic transaction scheduling
- Error detection and packet validation

Inputs:

- Sensor data packets from FPGA or HIL simulator
- Torque command outputs from control law

Outputs:

- Validated sensor data to estimator
- Formatted torque commands to FPGA
- Timing diagnostics to telemetry log

Interface Validation:

- Measure IMU packet arrival jitter.
- Measure torque command transmission latency.
- Verify timestamp step.

3.3.4 Deterministic Timing Requirements

Task	Frequency	Max Jitter
Sensor Acquisition	1 kHz	$< 50 \mu s$
State Estimation	1 kHz	$< 50 \mu s$
Control Law	1 kHz	$< 50 \mu s$
Torque Transmission	1 kHz	$< 50 \mu s$
Telemetry	100 Hz	$< 200 \mu s$

This phase concludes once the full estimator-control loop operates deterministically under RTOS scheduling and matches Python reference behavior when driven by identical sensor logs.

3.4 Phase 4: Full Hardware-in-the-Loop Integration

3.4.1 Integration Levels

- Python Closed Loop Validation
- FPGA + Python Hybrid Validation
- FPGA + HIL MCU Validation
- Full OBC + FPGA Closed Loop

3.4.2 Logged Signals

- Log A: Truth State
- Log B: Sensor Output
- Log C: Interface Timing (IMU)
- Log D: Estimated State
- Log E: Controller Output
- Log F: Interface Timing (Torque Command)

4 Interface Control Document (ICD)

4.1 IMU Packet Structure

Field	Type	Units
Timestamp	uint32	μs
Angular Rate X	int16	rad/s
Angular Rate Y	int16	rad/s
Angular Rate Z	int16	rad/s

4.2 Torque Command Packet

Field	Type	Units
Timestamp	uint32	μs
Torque X	int16	Nm
Torque Y	int16	Nm
Torque Z	int16	Nm

Note: ICD will continue to be updated.